Tecnológico de Monterrey

14. Modelos de clasificación

Ciencia de datos para la toma de decisiones II

Jorge Juvenal Campos Ferreira
juvenal.campos@tec.mx

Hoy vamos del texto crudo a un modelo evaluado

1. Del texto a la matriz

Matriz documento-término con cast_dtm() y con recetas de textrecipes. Por qué casi toda la matriz son ceros y cómo reducirla.

2. Etiquetas con un LLM

Claude clasifica 1,547 comentarios reales con una rúbrica escrita de antemano. Por qué la etiqueta es el techo del modelo.

3. Modelos supervisados

Partición entrenamiento/prueba, árboles de decisión, bosque aleatorio y regresión logística penalizada, todo con tidymodels.

4. Evaluación

Matriz de confusión, exactitud, precisión, sensibilidad, F1 y curva ROC. Por qué la exactitud engaña y qué palabras usa el modelo.

La matriz documento-término convierte texto en números

Cada fila es un documento (comentario, tweet, párrafo), cada columna es un término y cada celda guarda cuántas veces aparece el término en el documento, o su ponderación tf-idf.

Documento \ Término	impuesto	pinche	videojuego	gobierno	morena
Comentario 1	2	0	1	1	0
Comentario 2	0	1	0	1	1
Comentario 3	1	0	3	0	0

```
matriz_dtm <- tokens %>%
  count(id, palabra) %>%
  cast_dtm(document = id,
           term = palabra,
           value = n,
           weighting = tm::weightTfIdf)
```

tidytext::cast_dtm()

document: columna que identifica al documento.
term: columna con las palabras o tokens.
value: frecuencia o ponderación de la celda.
weighting: tf (conteo) o tf-idf.

La matriz es el puente: el modelo no lee texto, lee esta tabla.

En tidymodels no se construye a mano: una receta de textrecipes la produce dentro del flujo con step_tokenize(), step_stopwords(), step_tokenfilter() y step_tfidf().

Casi toda la matriz de texto son ceros

5,202 palabras
distintas aparecen en los comentarios de entrenamiento después de quitar las palabras vacías (stop words).

~17 palabras
usa el comentario mediano. Cada fila tiene valores en menos del 1 % de sus columnas.

500 columnas
quedan tras exigir que la palabra aparezca al menos 3 veces y conservar las 500 más frecuentes.

Por qué importa

Guardar millones de ceros desperdicia memoria y tiempo de cómputo. Un bosque con 4,000 columnas y 1,000 filas tarda mucho más y no aprende mejor: las palabras que aparecen una vez no sirven para generalizar.

Cómo reducirla

tm::removeSparseTerms(dtm, 0.99) elimina los términos ausentes en más del 99 % de los documentos. textrecipes::step_tokenfilter(min_times, max_tokens) hace lo mismo dentro de la receta y solo con la base de entrenamiento.

Cuánto recortar depende del tamaño del corpus y de la computadora; es una decisión del analista, no una regla.

Un LLM etiqueta miles de comentarios en minutos

Etiquetado con LLM

Asignar etiquetas a textos con un modelo de lenguaje de gran escala. En lugar de reglas o de anotadores humanos, se aprovecha la comprensión semántica del modelo para emitir un juicio contextual ("Ofensivo" / "No ofensivo").

Ejemplo real de la base

Rúbrica: "Clasifica el comentario como OFENSIVO o NO_OFENSIVO".
Texto: "Pues era porra en la UNAM".
Respuesta: NO_OFENSIVO.

Flujo técnico de la sesión (Claude + R)

Excel con 1,547 comentarios > **8 lotes de ~200 comentarios** > **Claude aplica la rúbrica** > **CSV con id y etiqueta** > **R une etiquetas y base (Excel)**

Un modelo grande y una rúbrica explícita

El criterio se escribe antes de ver los datos y se guarda con el código. Así cualquiera puede repetir el etiquetado con otro modelo (local con Ollama o por API) y comparar.

La rúbrica define qué cuenta como ofensivo

```
rubrica <- str_c(
  "Eres un moderador de comentarios de YouTube en español de México. ",
  "Clasifica el comentario como OFENSIVO solo si cumple al menos una: ",
  "(1) contiene insultos, groserías o apodos despectivos dirigidos a una ",
  "persona o grupo (incluye albures y modismos mexicanos); (2) ataca a ",
  "alguien por su clase social, género, orientación sexual, origen, edad o ",
  "apariencia; (3) desea daño, amenaza o humilla. Criticar con dureza, ",
  "burlarse de una idea o estar en desacuerdo con un gobierno, partido o ",
  "persona pública SIN insultos ni ataques personales es NO_OFENSIVO. ",
  "Comentarios neutros, de apoyo, chistes sin blanco humano o preguntas son ",
  "NO_OFENSIVO. Responde únicamente con una palabra: OFENSIVO o NO_OFENSIVO."
)
```

Ventajas

Rápido y flexible ante criterios nuevos.
No requiere datos de entrenamiento.
Cambiar el criterio es cambiar el prompt.
Se le puede pedir que explique su decisión.

Limitaciones

Varía entre intentos y entre modelos.
Hay que normalizar la salida (regex).
Hereda el sesgo del modelo y de la rúbrica.
Una rúbrica vaga produce etiquetas vagas.

La definición operativa de la rúbrica es la definición operativa del modelo que entrenaremos después.

Desde R, ellmer manda la rúbrica y cada comentario a Claude

```
library(ellmer)                  # un solo paquete para varias API de LLM

chat <- chat_anthropic(
  system_prompt = rubrica,          # la rúbrica de la diapositiva anterior
  model = "claude-fable-5-1",       # el modelo que etiquetó la base
  params = params(temperature = 0,   # respuestas reproducibles
                  max_tokens = 10)   # bastan pocos tokens
)
respuesta <- chat$chat(str_c("Comentario: \"", texto_i, "\""), echo = "none")

# Normalizar la salida: "OFENSIVO.", "no_ofensivo", "No ofensivo" ...
etiqueta <- case_when(
  str_detect(str_to_upper(respuesta), "NO[_ ]?OFENSIVO") ~ "No ofensivo",
  str_detect(str_to_upper(respuesta), "OFENSIVO")       ~ "Ofensivo",
  TRUE ~ NA_character_
)

# Con un modelo local basta cambiar el constructor:
chat <- chat_ollama(system_prompt = rubrica, model = "gemma4:12b")
```

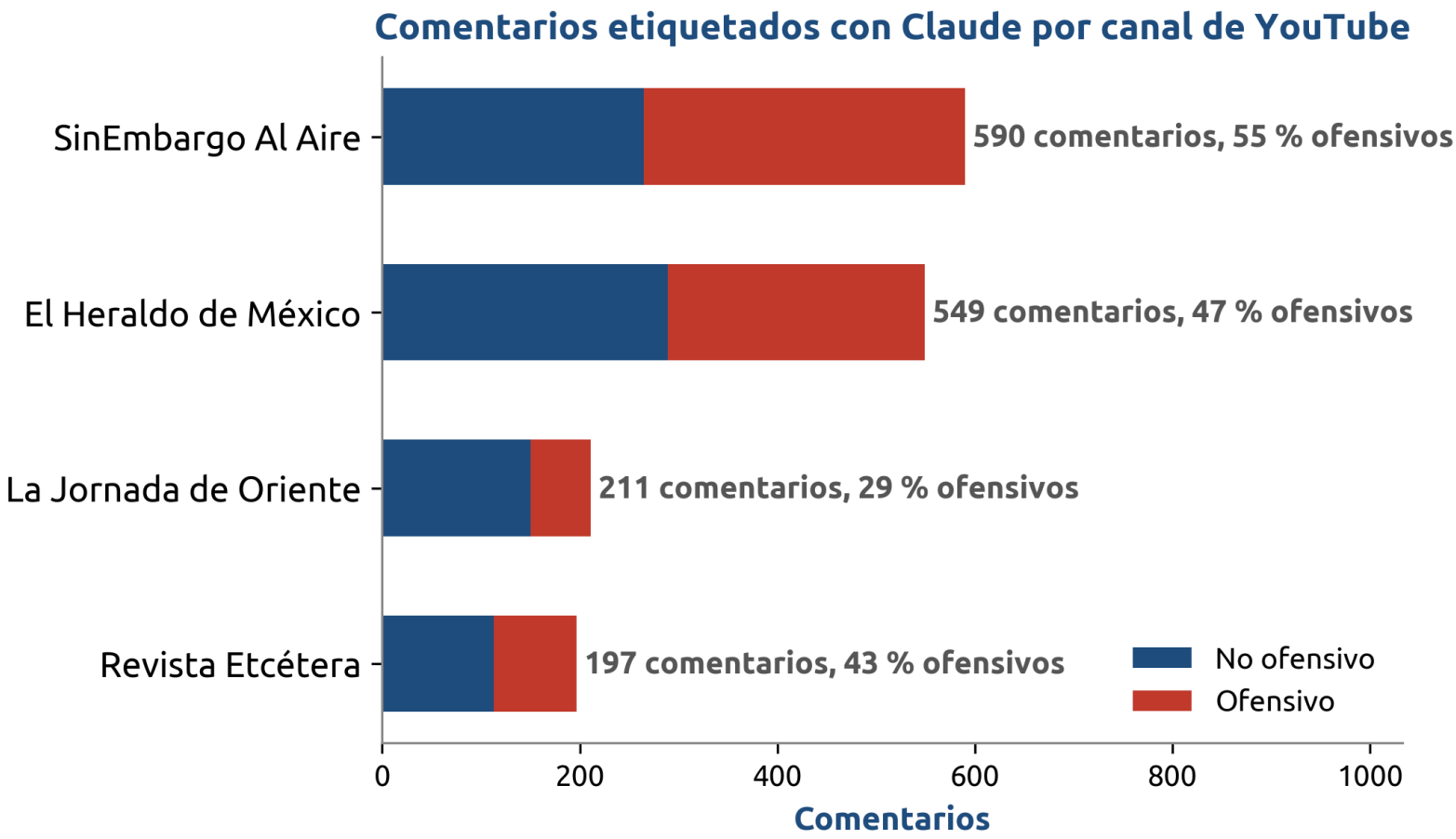
Cuidado con el orden

El patrón NO_OFENSIVO se prueba primero; si no, "no ofensivo" caería en "ofensivo".

La llave nunca va en el código

ANTHROPIC_API_KEY se guarda en .Renviron; ellmer la lee sola. Un modelo local (Ollama) no necesita llave.

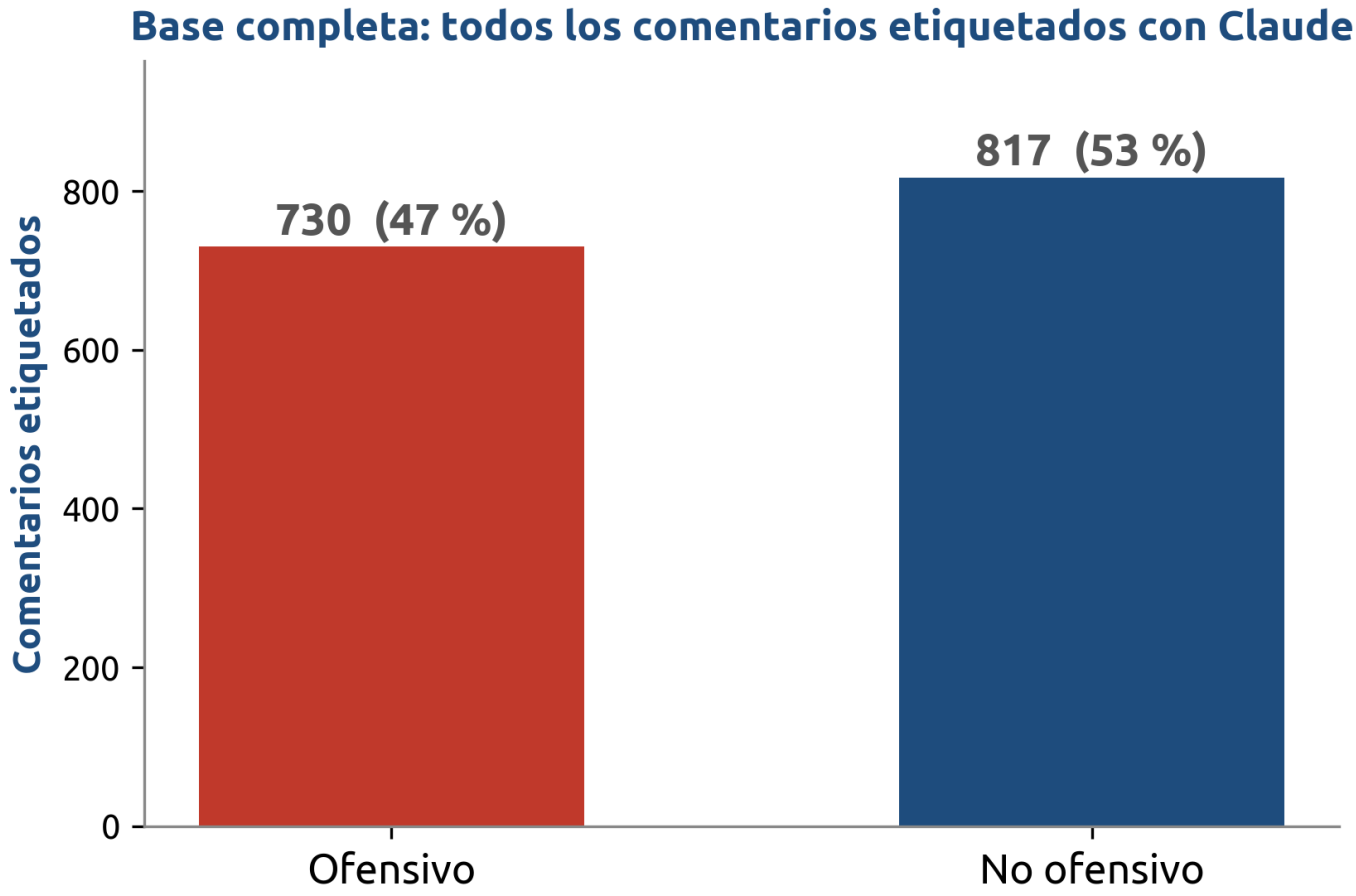
La proporción de ofensivos va de 29 % a 55 % según el canal



La misma rúbrica aplicada a públicos distintos da bases distintas. Un modelo entrenado con un canal no generaliza solo a otro.

Fuente: comentarios de YouTube a videos de El Heraldo de México, SinEmbargo, Revista Etcétera y La Jornada de Oriente (sep.-oct. 2026). Etiquetado con Claude (Fable 5.1). Cálculos propios.

La base tiene 1,547 comentarios etiquetados, 47 % ofensivos



La clase mayoritaria es "No ofensivo" (53 %). Ese desbalance va a importar al leer la exactitud.

Fuente: comentarios de YouTube a videos de El Heraldo de México, SinEmbargo, Revista Etcétera y La Jornada de Oriente (sep.-oct. 2026). Etiquetado con Claude (Fable 5.1). Cálculos propios.

Clasificar es aprendizaje supervisado

Modelado de clasificación

Entrenar un algoritmo para predecir la categoría de documentos nuevos a partir de patrones aprendidos en documentos ya etiquetados. Es una de las aplicaciones más prácticas del análisis de texto.

Características

Supervisado: necesita etiquetas.

Aprende la relación palabras -> clase.

Predice la clase de textos que nunca vio.

Se evalúa con datos que no usó para aprender.

Su techo es la calidad de las etiquetas.

Algunos modelos

Regresión logística (hoy, como base).

Árboles de decisión y bosques aleatorios (hoy).

XGBoost y otros ensambles.

Máquinas de soporte vectorial (SVM).

Redes neuronales y modelos tipo BERT.

Hoy: regresión logística y bosque aleatorio, los dos con el mismo flujo de tidymodels.

El flujo (partir, preparar, entrenar, evaluar) es el mismo para cualquier modelo; cambiar de algoritmo cuesta una línea.

Primero se aparta la base de prueba, antes de tocar nada

Entrenamiento: 75 % (1,159 comentarios) | **Prueba: 25 % (388)**

Por qué

Un modelo puede memorizar los datos con los que aprendió (sobreajuste). Solo los datos que nunca vio dicen qué tan bien generaliza.

Cómo

initial_split(prop = 0.75, strata = etiqueta) reparte al azar y conserva la proporción de clases en ambas bases. set.seed() hace el reparto reproducible.

Error frecuente en texto

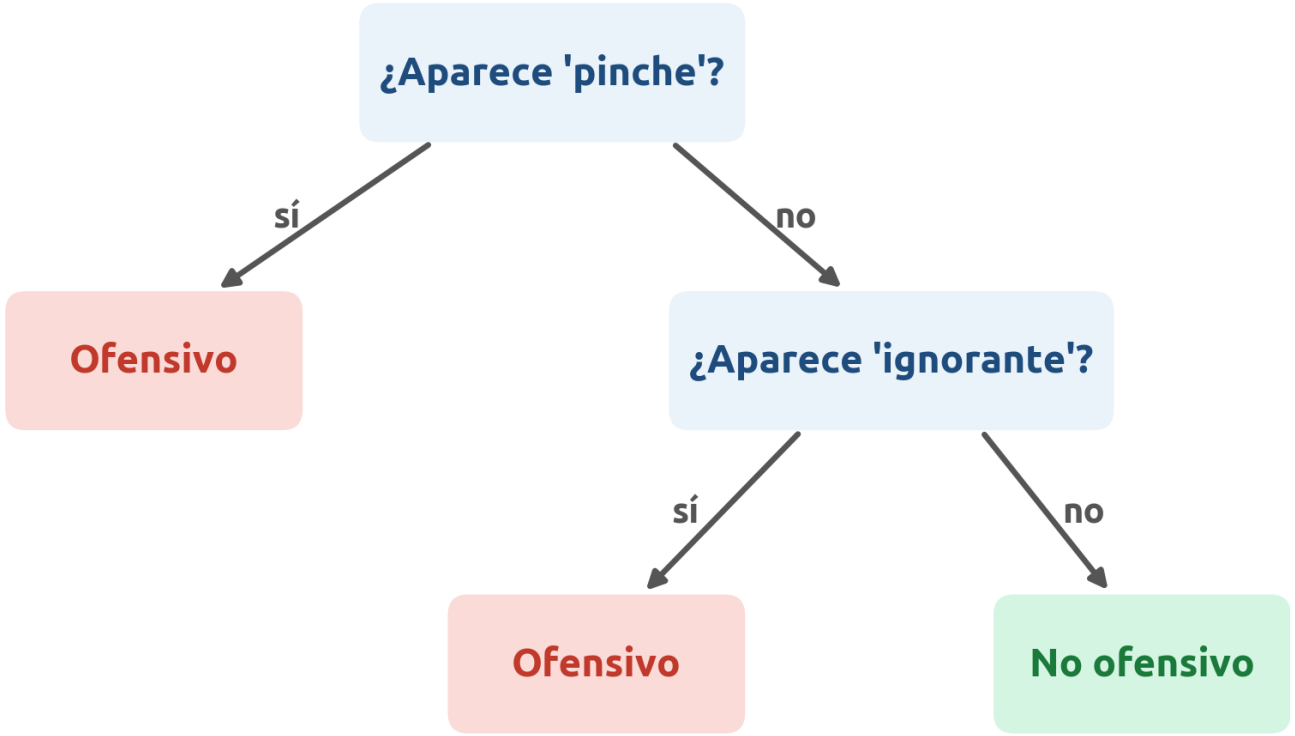
Construir la matriz documento-término con todos los comentarios y partir después. El vocabulario de prueba se filtra a la base de entrenamiento: fuga de información.

Lo que hace tidymodels

La receta se ajusta (prep) solo con entrenamiento. Al predecir la base de prueba, las palabras nuevas simplemente se ignoran.

```
set.seed(2026)
particion <- initial_split(comentarios, prop = 0.75, strata = etiqueta)
entrenamiento <- training(particion)
prueba        <- testing(particion)
```

Un árbol de decisión hace preguntas sobre palabras



Cada árbol del bosque ve una muestra distinta de comentarios y de palabras.
El bosque vota: gana la clase que predicen más árboles.

Cada pregunta divide los comentarios en dos grupos más puros. Un solo árbol memoriza; muchos árboles distintos generalizan.

El bosque aleatorio promedia cientos de árboles distintos

Muestras distintas

Cada árbol se entrena con una muestra con reemplazo de los comentarios (bagging). Los que no entran sirven para estimar el error "fuera de la bolsa" (OOB).

Palabras distintas

En cada división el árbol solo puede elegir entre mtry columnas al azar. Así ningún árbol depende de la misma grosería y el bosque es más diverso.

Votación

Cada árbol vota una clase; el bosque reporta la proporción de votos como probabilidad. Gana la clase con más votos.

Hiperparámetros que sí conviene conocer

Hiperparámetro	Qué controla	Valor típico en texto
trees (ntree)	Número de árboles	100 para explorar; 300-500 normal; 1,000 para exprimir
mtry	Columnas candidatas por división	raíz del número de columnas; con 500 columnas, ~22
min_n (nodesize)	Comentarios mínimos por hoja	5-10; valores altos simplifican el árbol

```
rand_forest(trees = 500, mtry = 25, min_n = 5) %>%
  set_engine("ranger", importance = "impurity") %>%
  set_mode("classification")
```

La regresión logística penalizada es el modelo base

$$P(\text{ofensivo}) = 1 / (1 + \exp(-(b0 + b1*x1 + \ldots + bk*xk)))$$

x_j es el tf-idf de la palabra j en el comentario; b_j > 0 empuja hacia "Ofensivo" y b_j < 0 hacia "No ofensivo".

Por qué empezar aquí

Ya la conocen de cursos anteriores. Es rápida, estable con pocas observaciones y entrega un coeficiente por palabra que se puede leer. Si un modelo complejo no la supera, algo anda mal con los datos, no con el modelo.

Penalización lasso

Con cientos de columnas y pocos cientos de filas, la regresión sin penalizar sobreajusta. La penalización L1 (mixture = 1) fuerza a cero los coeficientes de las palabras que no ayudan: selecciona variables sola.

```
logistic_reg(penalty = 0.01, mixture = 1) %>%
  set_engine("glmnet") %>%
  set_mode("classification")
```

penalty se puede afinar con validación cruzada (tune); hoy se fija en 0.01 para mantener el flujo simple.

La receta aprende el vocabulario solo con entrenamiento

```
library(tidymodels)
library(textrecipes)

stopwords_es <- stopwords::stopwords("es", source = "stopwords-iso")

receta <- recipe(etiqueta ~ texto, data = entrenamiento) %>%
  step_mutate(texto = str_remove_all(texto, "@@\\S+")) %>%  # menciones
  step_mutate(texto = str_remove_all(texto, "\\d+")) %>%    # dígitos
  step_tokenize(texto) %>%                                  # palabras
  step_stopwords(texto, custom_stopword_source = stopwords_es) %>%
  step_tokenfilter(texto, min_times = 3, max_tokens = 500) %>%
  step_tfidf(texto)                                         # DTM tf-idf

# Para inspeccionar la matriz que produce:
receta %>% prep() %>% bake(new_data = NULL) %>% dim()
#> [1] 1159 501
```

step_tokenize

Parte el texto en palabras. Acepta otros motores (n-gramas, lematización con udpipe).

step_tokenfilter

Filtra la dispersión: mínimo de apariciones y máximo de columnas, calculados en entrenamiento.

step_tfidf

Crea una columna tfidf_texto_palabra por término. Hace lo mismo que cast_dtm(weighting = weightTfIdf).

Receta + modelo = flujo de trabajo; el mismo para ambos

```
# Un flujo de trabajo une la receta con el modelo
flujo_bosque <- workflow() %>%
  add_recipe(receta) %>%
  add_model(modelo_bosque)

# Entrenar: prep de la receta + ajuste del modelo, en un paso
ajuste_bosque <- fit(flujo_bosque, data = entrenamiento)

# Predecir la base de prueba: augment agrega .pred_class y .pred_Ofensivo
predicciones <- augment(ajuste_bosque, new_data = prueba)

# Evaluar
predicciones %>% conf_mat(truth = etiqueta, estimate = .pred_class)
predicciones %>%
  metric_set(accuracy, precision, recall, f_meas)(
    truth = etiqueta, estimate = .pred_class)
predicciones %>% roc_auc(truth = etiqueta, .pred_Ofensivo)
```

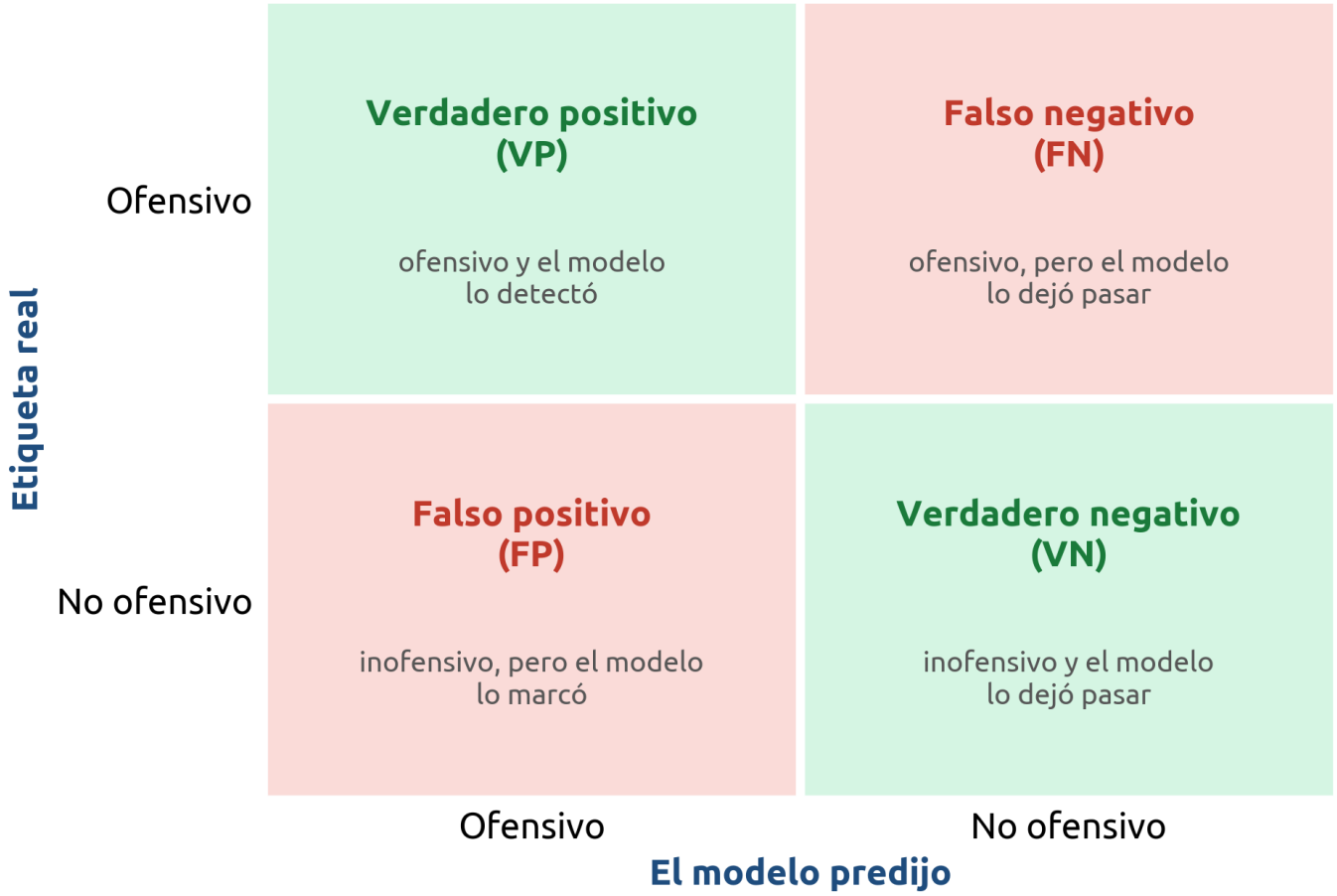
Cambiar de modelo = cambiar una línea

add_model(modelo_logistico) reutiliza la misma receta y la misma evaluación.

Clase positiva

yardstick toma el primer nivel del factor como positivo: por eso etiqueta se define con levels = c("Ofensivo", ...).

La matriz de confusión separa cuatro tipos de resultado



Para un moderador, el falso negativo (ofensa que pasa) y el falso positivo (crítica censurada) cuestan distinto. Las métricas miden cada uno.

Cuatro métricas responden cuatro preguntas distintas

Exactitud = (VP + VN) / total

¿Qué proporción de todos los comentarios clasificó bien? Engaña cuando una clase domina.

Precisión = VP / (VP + FP)

De los que marcó como ofensivos, ¿cuántos lo eran? Alta precisión = pocas críticas censuradas por error.

Sensibilidad = VP / (VP + FN)

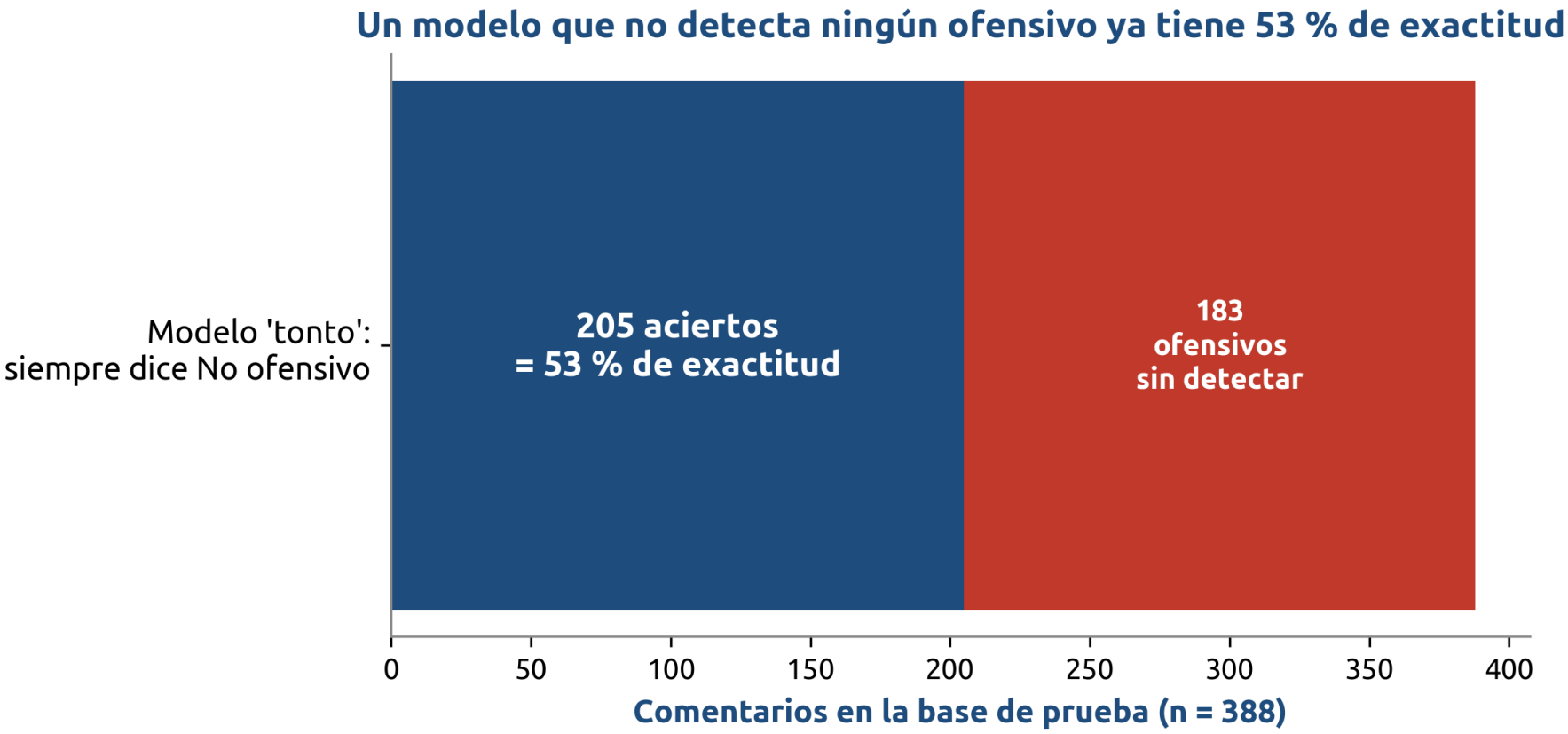
De los ofensivos reales, ¿cuántos detectó? Alta sensibilidad = pocas ofensas que se escapan. También se llama recall.

F1 = 2 * Prec * Sens / (Prec + Sens)

Media armónica de precisión y sensibilidad. Resume ambas en un número cuando no hay una preferida.

Siempre reportar al menos exactitud, precisión y sensibilidad de la clase que importa.

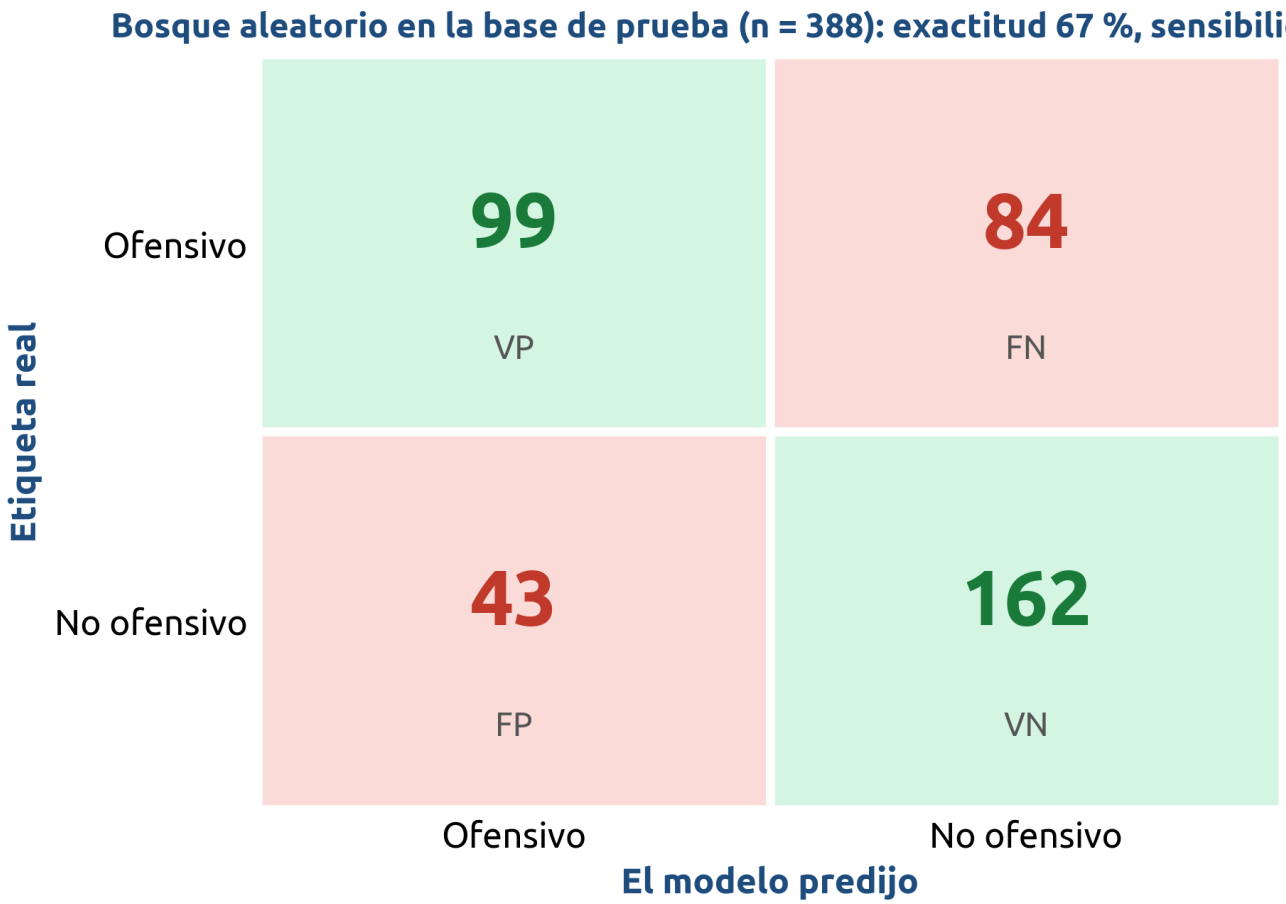
Un modelo que nunca detecta ofensas ya tiene 53 % de exactitud



Cualquier modelo debe compararse contra esa referencia de 53 %. Su sensibilidad es 0 %: no sirve para nada aunque su exactitud suene bien.

Fuente: comentarios de YouTube a videos de El Heraldo de México, SinEmbargo, Revista Etcétera y La Jornada de Oriente (sep.-oct. 2026). Etiquetado con Claude (Fable 5.1). Cálculos propios.

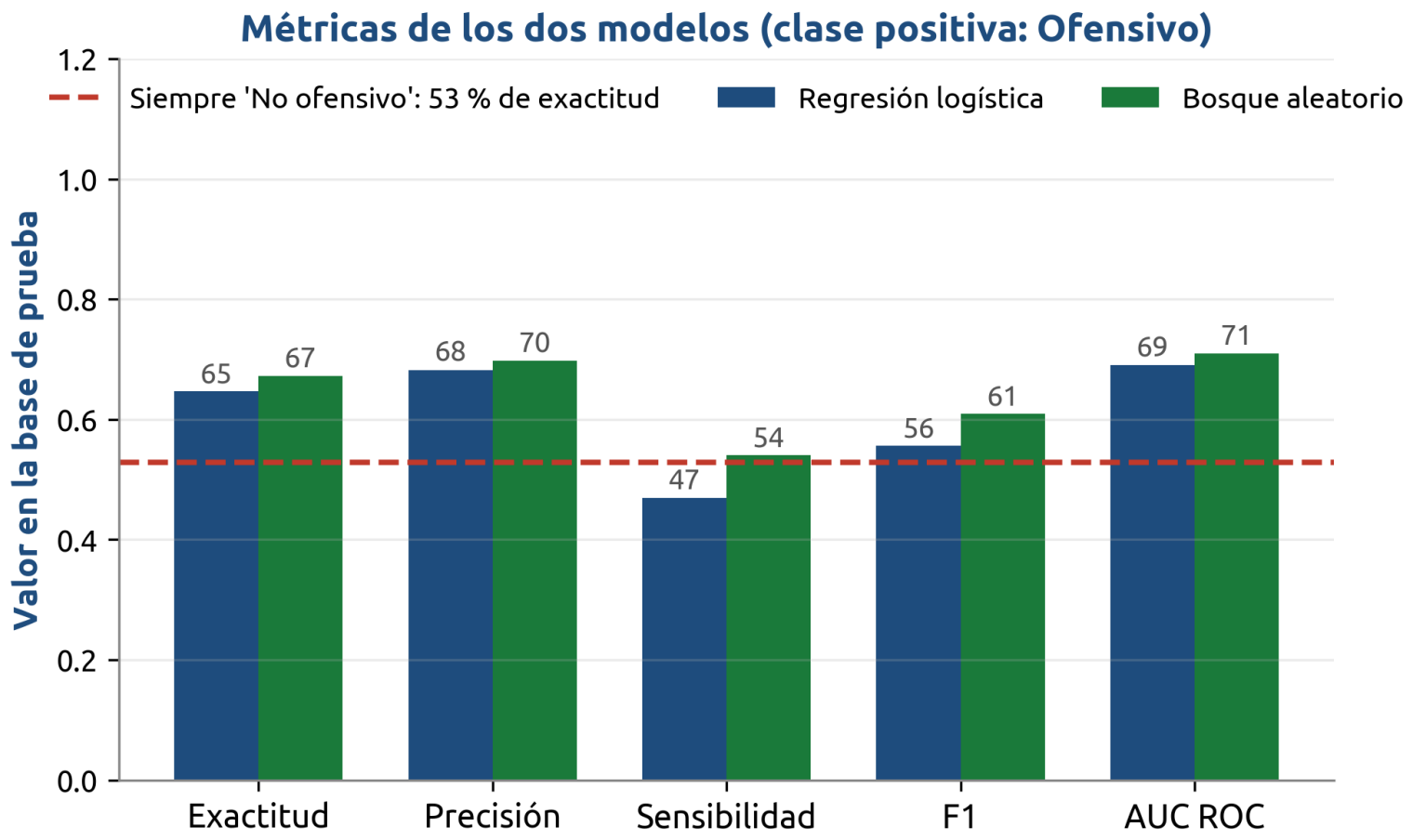
El bosque acierta 67 % y detecta 54 % de los ofensivos



Precisión de 70 %: cuando el bosque marca un comentario como ofensivo, acierta 70 de cada 100.

Fuente: comentarios de YouTube a videos de El Heraldo de México, SinEmbargo, Revista Etcétera y La Jornada de Oriente (sep.-oct. 2026). Etiquetado con Claude (Fable 5.1). Cálculos propios.

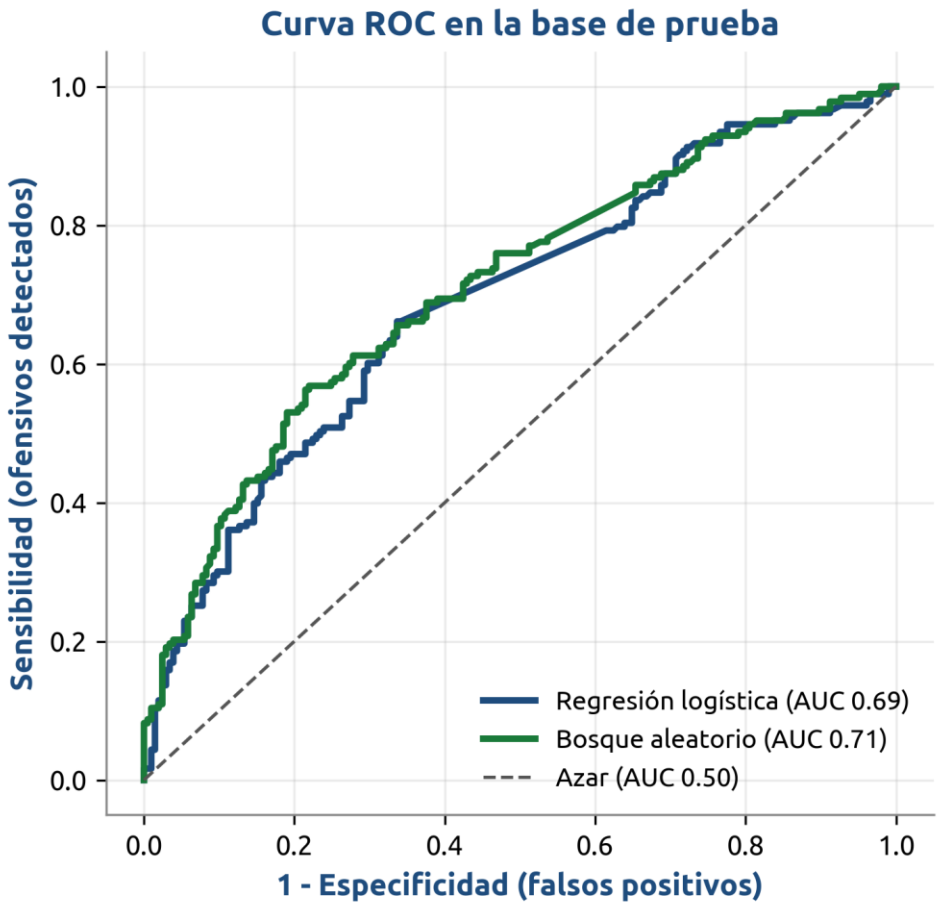
Bosque aleatorio obtiene el mejor F1; ambos superan la referencia de 53 %



Logística: exactitud 65 %, sensibilidad 47 %. Bosque: exactitud 67 %, sensibilidad 54 %. La ganancia sobre la referencia es el valor real del modelo.

Fuente: comentarios de YouTube a videos de El Heraldo de México, SinEmbargo, Revista Etcétera y La Jornada de Oriente (sep.-oct. 2026). Etiquetado con Claude (Fable 5.1). Cálculos propios.

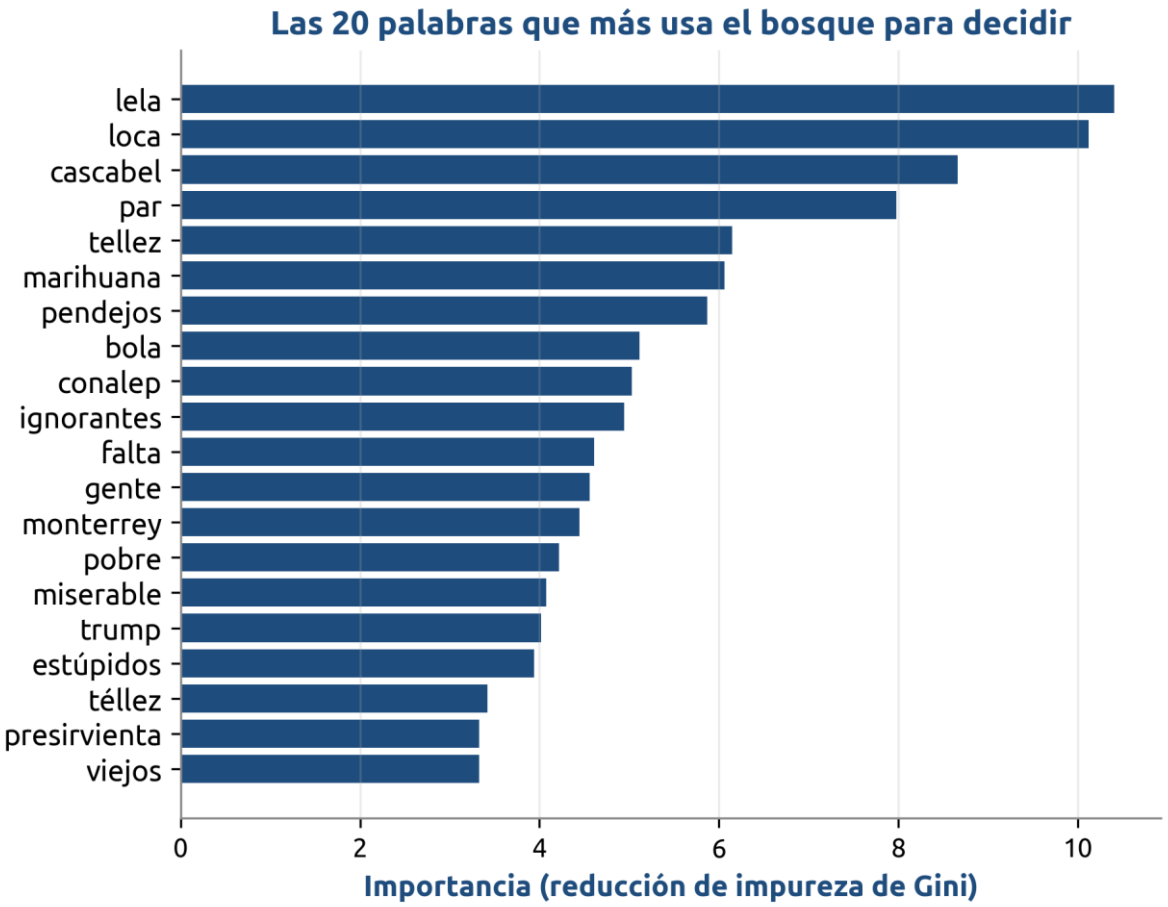
La curva ROC muestra el intercambio entre detectar y equivocarse



AUC de 0.71 (bosque) y 0.69 (logística). Mover el umbral de 0.5 cambia sensibilidad por falsos positivos sin reentrenar.

Fuente: comentarios de YouTube a videos de El Heraldo de México, SinEmbargo, Revista Etcétera y La Jornada de Oriente (sep.-oct. 2026). Etiquetado con Claude (Fable 5.1). Cálculos propios.

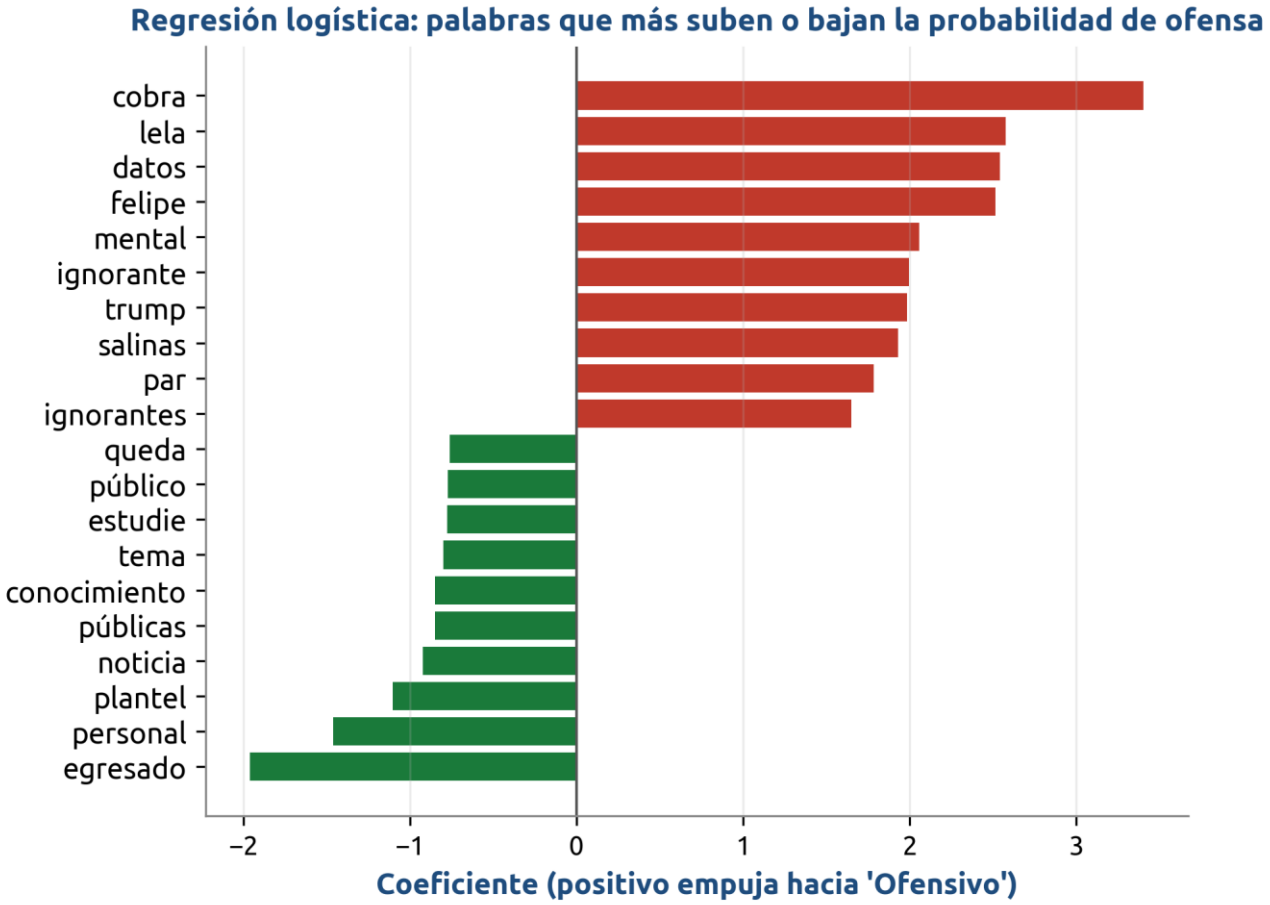
Las palabras importantes son groserías y apodos políticos



Encabezan la lista: lela, loca, cascabel, par, tellez, marihuana. El modelo aprendió el vocabulario del insulto, pero también el del tema: eso es señal de sesgo.

Fuente: comentarios de YouTube a videos de El Heraldo de México, SinEmbargo, Revista Etcétera y La Jornada de Oriente (sep.-oct. 2026). Etiquetado con Claude (Fable 5.1). Cálculos propios.

Los coeficientes de la logística muestran hacia dónde empuja cada palabra



A diferencia del bosque, el signo dice la dirección: rojo sube la probabilidad de ofensa, verde la baja. La lasso dejó en cero el resto.

Fuente: comentarios de YouTube a videos de El Heraldo de México, SinEmbargo, Revista Etcétera y La Jornada de Oriente (sep.-oct. 2026). Etiquetado con Claude (Fable 5.1). Cálculos propios.

Los errores del bosque enseñan sus límites

Comentarios de la base de prueba en los que el bosque se equivocó con más confianza:

Real: Ofensivo | Modelo: No ofensivo (P(ofensivo) = 0.06)

"Estúpida esta que esta hablando, le dio la crisis por las burradas de su gobierno, burradas de sus militantes, burradas de los excesos de dinero, y lastima eres inculta, estudia preguntale a un doctor, parque que tu burra apren..."

Real: Ofensivo | Modelo: No ofensivo (P(ofensivo) = 0.07)

"Y a Fox cuando lo señalaran? A la chupitos y a Calde-ron ?"

Real: Ofensivo | Modelo: No ofensivo (P(ofensivo) = 0.10)

"Y la otra también es la Sapa que no tiene para fregar alos verdaderos guerreros así es usted con los ojos de sapo vidriosos y lafivinosos"

Real: No ofensivo | Modelo: Ofensivo (P(ofensivo) = 0.89)

"Que los parrilleros le paguen beca de 100% y titulacion en el Tec de Monterrey ,campus Monterrey a estos jovenes, unos estudiamos donde podemos, otros prefieren hacer videos para denostar a quien ningun daño a hecho.🖤"

Ironía, ofensas sin groserías y etiquetas discutibles: ahí falla la bolsa de palabras.

El LLM y el modelo supervisado se complementan

	Etiquetado con LLM	Modelo supervisado
Datos etiquetados	No necesita	Cientos o miles
Velocidad	Segundos por texto	Miles de textos por segundo
Costo por texto	Cómputo (GPU) o API	Casi cero una vez entrenado
Reproducibilidad	Varía entre modelos e intentos	Determinista con set.seed()
Interpretabilidad	Puede explicar en lenguaje natural	Coeficientes e importancia de palabras
Cambiar el criterio	Reescribir el prompt	Reetiquetar y reentrenar
Ironía y contexto	Mejor	Débil (bolsa de palabras)
Dependencias	Servidor o API disponible	Solo R y el modelo guardado

El flujo real en la industria

El LLM etiqueta una muestra (o la etiquetan personas); con ella se entrena un modelo barato y rápido que clasifica millones de textos; el LLM revisa solo los casos dudosos.

El modelo mejora con mejores etiquetas, no solo con más árboles

1. Mejores etiquetas

Revisar a mano una muestra, pedir una segunda opinión a otro LLM o afinar la rúbrica. Es la palanca más grande: el modelo no supera el ruido de sus etiquetas.

2. Más datos

Comentarios de más videos y temas. Con 1,000 filas los intervalos de las métricas son anchos.

3. Mejores variables

Diccionario de groserías como columna extra, bigramas, lematización con udpipe, conteo de mayúsculas o signos de exclamación.

4. Afinar hiperparámetros

tune_grid() con validación cruzada para penalty, mtry y min_n. Nunca afinar sobre la base de prueba.

5. Otro modelo o ensamble

XGBoost, SVM o un modelo tipo BERT en español. Cambiar add_model() y comparar con las mismas métricas.

6. Mover el umbral

Si importa más no dejar pasar ofensas, bajar el umbral de 0.5 sube la sensibilidad a costa de la precisión.

Orden sugerido: etiquetas, datos, variables; los hiperparámetros al final.

Un clasificador hereda los sesgos de quien etiqueta

Lo que aprendió el bosque

Entre las palabras más importantes aparecen nombres y apodos políticos. El modelo asocia el tema con la ofensa, no solo el insulto. Un comentario crítico sin groserías puede quedar marcado.

Español coloquial

"Wey", "pinche" o "neta" aparecen en comentarios amistosos. Un modelo entrenado con etiquetas estrictas penaliza registros informales y a quienes los usan.

Preguntas antes de usarlo

¿Quién definió "ofensivo" y con qué criterio?
¿Qué error cuesta más: censurar o dejar pasar?
¿Se evaluó por tema, por grupo o solo en promedio?
¿Hay revisión humana para los casos dudosos?

Buenas prácticas

Documentar el prompt o la guía de anotación.
Reportar precisión y sensibilidad por subgrupo.
Mantener a una persona en el circuito de decisión.
Reentrenar cuando cambie el tema o el lenguaje.

El modelo describe las etiquetas; no define qué es ofensivo.

Conclusiones

Cinco ideas para llevarse

1. La matriz documento-término es el puente entre texto y modelo; es dispersa y hay que recortarla con criterio.

2. Un LLM etiqueta rápido con una rúbrica explícita, pero la etiqueta es el techo del modelo: hay que auditarla.

3. La base de prueba se aparta antes de construir el vocabulario; tidymodels lo garantiza con recetas y flujos de trabajo.

4. La exactitud engaña: la referencia era 53 %. Se reportan precisión, sensibilidad y F1 de la clase que importa.

5. Las palabras importantes revelan qué aprendió el modelo, incluidos sus sesgos. Mejorar etiquetas rinde más que afinar hiperparámetros.

Recursos

Hvitfeldt, E. y Silge, J. (2021). Supervised Machine Learning for Text Analysis in R. CRC Press.

Libro gratuito en línea: smltar.com. Capítulos 5 a 7 cubren exactamente el flujo de hoy.

Silge, J. y Robinson, D. (2017). Text Mining with R: A Tidy Approach. O'Reilly.

tidytextmining.com. Capítulo 5: cast_dtm() y matrices dispersas.

Kuhn, M. y Silge, J. (2022). Tidy Modeling with R. O'Reilly.

tmwr.org. Recetas, flujos de trabajo, yardstick y tune.

Documentación de textrecipes y ellmer

textrecipes.tidymodels.org; ellmer.tidyverse.org (chat_anthropic, chat_ollama).

Código y datos de la sesión

02_Codigo/Sesion_14_Clasificacion: 01_etiquetado_claude.R, 02_modelo_tidymodels.R y la carpeta 01_datos.

Gracias

¿Preguntas?